# 10

# Design a Tic Tac Toe Game

In this chapter, we will explore the classic interview question of designing a Tic Tac Toe game. We aim to create an interactive platform where players can engage in real-time gameplay, with the ability to track game states, manage turns, and display the game's components such as the grid, scores, move tracking, system interactions, and overall gameplay.

The goal is to design a Tic Tac Toe game that supports two-player gameplay, ensuring a fair and quick player selection process. The game's UI will feature a 3x3 grid where each cell can be marked with either "X" or "O" and select turns in an orderly manner. The game is to ship three identical systems functionally, vertically, or diagonally. The system should efficiently manage the game state, ensuring that the game progresses smoothly and accurately.

Let's gather the key requirements through clarifying questions and discussions with the interviewer.

## Requirements Gathering

Here's an example of a typical prompt an interviewer might introduce:

"Imagine you're tasked with designing a system for a popular game like Tic Tac Toe. You want choices a system (e.g., "X" or "O") and take turns placing their symbols on a grid. The game should automatically detect when a player wins by forming a line of three symbols in a row, column, or diagonal. Behind the scenes, the game tracks your moves, updates a scoreboard in real-time, and ensures fair gameplay. Your task is to design a system that supports this game. What questions would you ask to clarify the requirements?"

## Requirements clarification

Here's an example of how a conversation between a candidate and an interviewer might unfold:

**Candidate:** Does the system support different board sizes?

**Interviewer:** No, let's stick to the standard 3x3 gameplay.

**Candidate:** How should the game handle customizations like skins, themes, and sounds?

**Interviewer:** Focus on the core gameplay mechanics and functionality for now, not skins, or skipping.

**Candidate:** Should the game track player statistics?

**Interviewer:** Yes, the system should track the number of wins and losses for each player, stored in a simple format.

**Candidate:** How does the game handle invalid moves?

**Interviewer:** If a player attempts a move on an occupied or invalid position, notify them and prompt them for a new move.

Based on the discussion, let's next down the key functional requirements.

## Requirements

Here are the key functional requirements we've identified:

- The game is played on a 3x3 board.
- Two players take turns placing their symbols.
  - A player chooses a position (empty) to place a mark, or skipping.
  - A player (or board) wins or ended.
  - Skipping is allowed.
- A score tracker records player performance, updates ratings based on wins, and supports queries like rankings or top players.
- Invalid moves (e.g., placing a symbol on an occupied cell) are rejected with feedback to the player.

Below are the non-functional requirements:

- The user interface should be intuitive, providing clear feedback for invalid moves and game outcomes, with easily accessible scoreboard information.
- The system should support future enhancements, such as different board sizes or game modes, without major architectural changes.

## Identify Core Objects

Before diving into the design, it's important to identify the core objects.

**Board:** The board represents the 3x3 grid where players place their symbols (e.g., "X" or "O"). It handles operations to place the grid, check for a winner, by maintaining the current state, and displaying, and determines if the board is full.

**Cells:** The cells represent individual spots on the board. Each cell can be empty or contain a player's symbol (e.g., "X" or "O").

**Game:** The game controls the overall flow of the game. It initializes new games, manages player turns, processes moves, checks for a winning, and ends the game's status, whether it's in progress or ended with a winner or draw.

**Design choices:** The Game class is chosen over a standalone function because it handles multiple operations. To keep it manageable, we delegate the Board to manage the grid and ScoreTracker to handle player statistics. This modularity enhances maintainability and scalability.

**ScoreTracker:** Tracks player statistics, such as wins and losses, ensuring accurate performance data.

Now that we've identified the core objects, let's design their relationships in a class diagram.

## Design Class Diagram

In this class diagram, we illustrate the relationships between the core objects in the Tic Tac Toe game. The Game is created a criterion design that adheres to SOLID principles, such as the Single Responsibility principle, ensuring each component handles a distinct aspect of the game. The Board manages the grid, the ScoreTracker handles player statistics, and the Game orchestrates the overall flow. This modular approach not only enhances maintainability and scalability but also facilitates easy testing and future enhancements.

## Board

The Board class represents the 3x3 game grid, which is modeled as a two-dimensional array of Player objects. It is responsible for enforcing game rules like placing marks, checking if the board is full, and determining if a player has won. It determines if a player has won by checking if the winning condition is met (e.g., column, or diagonal). It provides functionality to reset the grid for a new game and to retrieve the current state of the grid, which is essential for rendering the UI.

Here's the design of the class:

**Design choices:** The choice to include only checking logic within the Board class, rather than the Game class, ships with Single Responsibility Principle (SRP), as the Board is the owner of all grid related tasks.

## ScoreTracker

The ScoreTracker class is designed to monitor player performance across multiple games by maintaining a centralized scoreboard. It records wins and losses for each player, ensuring accurate tracking of game outcomes. The class provides methods to update player performance statistics and to retrieve a sorted leaderboard based on win rates. The leaderboard ranks players by their win rate, calculated as the number of wins divided by the total number of games, keeping the logic straightforward and effective.

Instead of storing raw data, such as individual game results, the ScoreTracker class manages the responsibility of ScoreTracker. This design choice reduces the overhead of storage, while a player's name (as the relevant field), ratings and contextual, refreshing performance metrics. The class uses efficient data structures, such as hash maps, to quickly access and update player records, ensuring that it can handle a large number of players and games without performance issues. This approach ensures that the system remains responsive and scalable as the number of players and games increases.

Here's the representation of this class:

## Complete Class Diagram

Below is the complete class diagram of the Tic Tac Toe game:

Complete Class Diagram of Tic Tac Toe

Let's continue to design a Tic Tac Toe game in Object-Oriented Programming.

## Code - Tic-Tac-Toe Game

In this section, we'll implement the core functionalities of the Tic Tac Toe game. Focusing on key areas such as managing the game board, handling player moves, checking for winning conditions, and tracking player statistics, we'll ensure the game runs smoothly and efficiently.

The Game class manages the flow of a Tic Tac Toe game. It manages the game board, player turns, and score tracker updates. After each move, the game checks for win conditions (three matching symbols in a row, column, or diagonal) and for a draw (when the board is full). Key methods include:

Check out the code implementation of the Game class below:

```
public class Game {
    // Game class to manage the game flow and state
    private Board board;
    private ScoreTracker scoreTracker;
    private int[] lastMove;

    public Game(int size, int[] players) {
        // Initialize the game with a board of size x size and the specified players
        this.board = new Board(size);
        this.scoreTracker = new ScoreTracker();
        this.lastMove = null;
    }

    // Check if a move is valid for the given player
    public boolean canMakeMove(int[] Player) {
        return !board.isFull();
    }

    // Make a move at the specified position for the given player
    public boolean makeMove(int row, int col, Player player) {
        if (!board.isValidPosition(row, col)) {
            // Invalid position
            System.out.println("Invalid position.");
            return false;
        }

        // Try to place the mark on the board
        boolean moveMade = board.placeMark(row, col, player);
        if (moveMade) {
            // Update last move if the move was successful
            this.lastMove = new int[]{row, col};
        }

        return moveMade;
    }

    // Check if there is a winner
    public int[] getWinner(int[] Player) {
        int winner = board.checkWinner();
        if (winner != 0) {
            int[] loser = (winner == 1) ? new int[]{2} : new int[]{1};
            scoreTracker.recordWinner(player, loser);
            return winner;
        }
        return null;
    }

    // Check if the game is over (either someone won or the board is full)
    public boolean isGameOver(int[] Player) {
        int winner = board.checkWinner();
        if (winner != 0 || board.isFull()) {
            int[] loser = (winner == 1) ? new int[]{2} : new int[]{1};
            if (winner == 0) {
                scoreTracker.recordDraw(player, loser);
            }
            return true;
        }
        return false;
    }

    // Get the player's score
    public int getPlayerScore(int[] Player) {
        return scoreTracker.getScore(player);
    }

    // Get the last move made
    public int[] getLastMove(int[] Player) {
        return lastMove;
    }
}
```

- **canMakeMove(int[] Player):** This method checks if the game is ongoing by ensuring the board isn't full. If the board is full, no more moves can be made, so the method returns false. Otherwise, it returns true, allowing the game to continue.
- **makeMove(int row, int col, Player player):** This method attempts to place a mark on the board at the specified row and column for the given player. It first checks if the move is valid using the board's isValidPosition method. If the move is valid, it places the mark on the board and updates the last move if the move results in a win or a draw, the score tracker is updated.
- **getWinner(int[] Player):** This method checks for a winner by calling the board's checkWinner method. If a winner is found, it updates the score tracker and returns the winner's information. If no winner exists, the method returns null.
- **isGameOver(int[] Player):** This method checks if the game is over, either by a win or a draw. It uses the board's checkWinner method to determine if a winner exists and the board's isFull method to check if the board is full.
- **getPlayerScore(int[] Player):** This method retrieves the score for the specified player from the score tracker.
- **getLastMove(int[] Player):** This method returns the last move made by the specified player.

**Implementation choices:** The Board class is 3D array (Player[][]) to represent the 3x3 grid, chosen for its direct mapping to the game's layout and efficient access and modification of grid cells.

## Player

The Player class represents a player in the game, with key attributes like their name and symbol. These attributes help identify the player and their marks on the board.

```
public class Player {
    // Player class to manage player data.
    private char symbol;

    public Player(String name, char symbol) {
        this.name = name;
        this.symbol = symbol;
    }

    public String getName() {
        return name;
    }

    public char getSymbol() {
        return symbol;
    }
}
```

## ScoreTracker

The ScoreTracker class manages player rankings across multiple Tic-Tac-Toe games by tracking and updating scores for a group of players. It uses a HashMap<Player, Integer> to store each player's rating, ensuring quick access and updates. The class provides methods to record wins and draws, adjusting player ratings accordingly, and to retrieve the top players based on their ratings.

```
class ScoreTracker {
    // ScoreTracker class to manage player ratings.
    private HashMap<Player, Integer> playerRatings;

    public ScoreTracker() {
        // Constructor to initialize the HashMap.
        this.playerRatings = new HashMap<>();
    }

    // Record a win for the winner and a loss for the loser.
    public void recordWinner(Player winner, int[] loser) {
        int winnerRating = playerRatings.getOrDefault(winner, 0);
        int loserRating = playerRatings.getOrDefault(loser, 0);
        playerRatings.put(winner, winnerRating + 1);
        playerRatings.put(loser, loserRating - 1);
    }

    // Record a draw.
    public void recordDraw(Player player1, Player player2) {
        int rating1 = playerRatings.getOrDefault(player1, 0);
        int rating2 = playerRatings.getOrDefault(player2, 0);
        playerRatings.put(player1, rating1 + 1);
        playerRatings.put(player2, rating2 + 1);
    }

    // Get the top players.
    public Map<Player, Integer> getTopPlayers(int[] Player) {
        List<Map.Entry<Player, Integer>> sortedPlayers = new ArrayList<>(playerRatings.entrySet());
        sortedPlayers.sort(Map.Entry.comparingByValue().reversed());
        int topN = 3;
        Map<Player, Integer> topPlayers = new LinkedHashMap<>();
        for (int i = 0; i < topN && i < sortedPlayers.size(); i++) {
            Map.Entry<Player, Integer> entry = sortedPlayers.get(i);
            topPlayers.put(entry.getKey(), entry.getValue());
        }
        return topPlayers;
    }

    // Get the rank of a player.
    public int getRankOf(Player player) {
        List<Map.Entry<Player, Integer>> sortedPlayers = new ArrayList<>(playerRatings.entrySet());
        sortedPlayers.sort(Map.Entry.comparingByValue().reversed());
        int rank = 1;
        for (Map.Entry<Player, Integer> entry : sortedPlayers) {
            if (entry.getKey().equals(player)) {
                return rank;
            }
            rank++;
        }
        return -1;
    }
}
```

- **recordWinner(Player winner, int[] loser):** This method records a win for the winner and a loss for the loser. It retrieves the current ratings of the winner and the loser from the HashMap, updates their ratings by adding 1 to the winner's rating and subtracting 1 from the loser's rating, and stores the updated ratings back in the HashMap.
- **recordDraw(Player player1, Player player2):** This method records a draw between two players. It retrieves the current ratings of both players from the HashMap, adds 1 to each player's rating, and stores the updated ratings back in the HashMap.
- **getTopPlayers(int[] Player):** This method returns the top players based on their ratings. It sorts the HashMap entries by value in descending order, takes the top 3 players (or fewer if there are less than 3), and returns them in a LinkedHashMap to maintain the order.
- **getRankOf(Player player):** This method returns the rank of a specified player. It sorts the HashMap entries by value in descending order, iterates through the sorted list to find the player, and returns their rank (starting from 1). If the player is not found, it returns -1.

**Implementation choices:** The ScoreTracker uses a HashMap<Player, Integer> to store player ratings, which is chosen for its O(1) average time access and update performance, making it ideal for frequent rank updates and queries (e.g., top ranked player).

## Deep Dive Topic

At this point, you have met the basic requirements of the question. In this section, we will explore some potential extensions to the system design.

## Implement undo functionality in Tic-Tac-Toe

Implementing undo functionality in the Tic-Tac-Toe game enhances user experience by allowing players to correct mistakes or explore different strategies. The Command design pattern is ideal for this, as it encapsulates each move as a command object, enabling easy reversal of actions by maintaining a history of executed commands.

**Step 1: Track move history**

- Every time a player makes a move, we store that move so it can be undone later.
- The Board class (which will be already designed) will keep track of the last game's state before the player's move, including details such as coordinates, symbols, and the player.

Below is the representation of the Move class:

**Step 2: Store Moves with a Stack**

- Moves are stored in a stack (LIFO). The last move is at the top, so it's easy to undone first, we can use a stack (ArrayDeque interface).
- Each move is pushed onto the stack when it's made.
- When undo is requested, we pop the last move from the stack and revert the board to the previous state.

Below is the representation of the Mementoes class, which records the moves and implements the undo functionality:

Here's the code for the Mementoes class:

```
// Mementoes class to manage move history.
class Mementoes {
    // Stack (LIFO) structure to store moves.
    private ArrayDeque<Move> history;

    // Constructor to initialize the history.
    public Mementoes(Board board) {
        this.history = new ArrayDeque<>();
    }

    // Add a new move to the history.
    public void addMove(Move move) {
        history.push(move);
    }

    // Undo the last move.
    public void undoLastMove(Board board) {
        if (!history.isEmpty()) {
            Move lastMove = history.pop();
            int[][] boardState = board.getBoardCopy();
            boardState[lastMove.getRowIndex()][lastMove.getColIndex()] = 0;
            board.setBoardState(boardState);
        }
    }
}
```

- **addMove(Move move):** This method adds a new move to the history by pushing it onto the stack.
- **undoLastMove(Board board):** This method retrieves the last move from the history, removes it from the board by setting the corresponding cell to 0 (empty), and updates the board state.

**Step 3: Reverse the Board State**

- The board state is then updated to reflect the undone move.
- This ensures the game can continue from the previous state.

Here's the updated code for the Game class:

```
public void undoMove(int[] Player) {
    // Get the last move from the history.
    int[] lastMove = mementoes.getLastMove();
    if (lastMove != null) {
        int rowIndex = lastMove[0];
        int colIndex = lastMove[1];

        // Set the cell to 0 (empty) on the board.
        int[][] boardState = board.getBoardCopy();
        boardState[rowIndex][colIndex] = 0;
        board.setBoardState(boardState);

        // Update the last move in the game.
        this.lastMove = null;

        // Update the score tracker.
        scoreTracker.recordWinner(Player, loser);
        scoreTracker.recordDraw(Player, Player);

        // Update the game state.
        int[] winner = getWinner(Player);
        if (winner != null) {
            int[] loser = (winner == 1) ? new int[]{2} : new int[]{1};
            scoreTracker.recordWinner(Player, loser);
        } else {
            scoreTracker.recordDraw(Player, Player);
        }
    }
}
```

- **Check the last move:** We first check if there is a last move to undo.
- **Update the board:** The board state is updated to reflect the undone move.
- **Update the last move:** The last move is set to null after it has been undone.
- **Update the score tracker:** The score tracker is updated to reflect the change in the game state.
- **Update the game state:** The game state is updated to reflect the undone move.

© 2025 Daily Dev Tips

Undo functionality using the memento pattern

However, one potential challenge of using the Memento Pattern is memory overhead. Every time a move is made, we store the entire move in the history. In Tic-Tac-Toe, this isn't much of a problem because the board is small, but in more complex games with larger states, the memory required to store each memento could become a bottleneck.

With our Tic-Tac-Toe game designed and implemented, let’s wrap up with key takeaways.

# Wrap Up

In this chapter, we designed the Tic-Tac-Toe game. We started by gathering and clarifying the requirements through a series of questions and answers. Next, we identified the core objects involved, designed the class diagram, and implemented the key components of the game.

A key takeaway from this design is the importance of modularity and clear separation of concerns. Each component, such as the Board, Game, Player, and ScoreTracker classes, focuses on a specific responsibility, ensuring the system is maintainable and easy to extend.

In the deep dive section, we explored advanced topics, including using the Memento Pattern to handle undo functionality, enabling players to revert their moves while maintaining the integrity of the game state.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Mark as Complete

**Partner With Us**

Be an Affiliate

Become a Contributor

**Company & Legal**

Our Team

Newsletter

Privacy Policy

Terms of Service

Do Not Sell or Share My Personal Information

**Support**

hi@bytebytego.com

Report a Bug

**Resources**

YouTube

Visual Dev Guides

Find Jobs

Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.